POO : Rappel des Bases de Java

Définition d'une structure de classe Java

Une classe est un plan pour créer des objets et définit l'état (attributs) et le comportement (méthodes) des objets. Structure de base :

```
public class NomDeLaClasse {
    // Variables d'instance
    private String nom;
    // Constructeurs
    public  NomDeLaClasse(String nom){
        this.nom = nom}
    // Méthodes
    public void maMethode(){
        //code source de traitement de donnée
    }
}
```

Les classes peuvent contenir des variables, des méthodes, des constructeurs, des blocs, des classes internes et des interfaces.

Qu'est-ce qu'un Constructeur en Java ?

Un constructeur est une méthode spéciale d'une classe qui est appelée lorsqu'un nouvel objet (une instance) de cette classe est créé. Son rôle principal est d'initialiser les variables d'instance de l'objet.

Caractéristiques d'un Constructeur

- Nom : Le nom du constructeur doit être le même que le nom de la classe.
- Pas de type de retour : Les constructeurs ne retournent pas de valeur et ne spécifient pas de type de retour, pas même void.
- Automatiquement appelé : Il est appelé automatiquement lors de l'instanciation d'un objet.
- Peut être surchargé : Une classe peut avoir plus d'un constructeur, chacun ayant des paramètres différents.

À quoi sert un Constructeur ?

- Initialisation d'Objet : Le constructeur permet d'initialiser l'état d'un objet lors de sa création. Cela peut inclure l'initialisation des variables d'instance avec des valeurs spécifiques.
- Réduction de la Redondance : Le code d'initialisation commun à tous les objets d'une classe peut être placé dans un constructeur, réduisant ainsi la redondance.

Création d'un Constructeur

Pour créer un constructeur, vous déclarez une méthode avec le même nom que la classe, sans type de retour.

```
public class Voiture {
    private String marque;

    // Constructeur de la classe Voiture
    public Voiture(String marque) {
        this.marque = marque; // Initialisation de la variable d'instance
marque
    }
}
```

Utilisation d'un Constructeur

Pour utiliser un constructeur, vous utilisez le mot-clé `new` suivi du nom de la classe et des paramètres appropriés.

```
public class Main {
    public static void main(String[] args) {
        Voiture maVoiture = new Voiture("Toyota"); // Création d'un objet
Voiture
    }
}
```

Dans cet exemple, un objet `Voiture` est créé en appelant le constructeur `Voiture(String marque)` avec le paramètre **"Toyota"**.

Constructeur Par Défaut

Si aucun constructeur n'est explicitement défini dans une classe, Java fournit un constructeur par défaut sans paramètre qui ne fait rien de spécial. Il initialise les variables d'instance avec des valeurs par défaut (null, 0, false, etc.).

Constructeur Sans Argument

Un constructeur sans argument est un constructeur qui ne prend aucun paramètre. Il est utile pour initialiser un objet avec des valeurs par défaut.

Surcharge de Constructeur

La surcharge de constructeur signifie avoir plus d'un constructeur dans une classe, chacun ayant un ensemble différent de paramètres (types ou nombre de paramètres).
Cela permet d'initialiser des objets d'une classe de différentes manières.

Exemple

Imaginons une classe `Personne` qui a deux variables d'instance : `nom` et `age`. Nous allons créer plusieurs constructeurs pour cette classe afin d'initialiser les objets `Personne` de différentes façons.

```
public class Personne {
    private String nom;
    private int age;

    // Constructeur sans argument
```

```
    public Personne() {
        this.nom = "Inconnu";
        this.age = 0;
    }

    // Constructeur avec un seul argument
    public Personne(String nom) {
        this.nom = nom;
        this.age = 0;
    }

    // Constructeur avec deux arguments
    public Personne(String nom, int age) {
        this.nom = nom;
        this.age = age;
    }
}
```

Dans cet exemple, la classe Personne a trois constructeurs :

Constructeur Sans Argument :

- Initialise le nom à "Inconnu" et age à 0.
- Utilisé quand on n'a pas d'informations spécifiques sur la personne au moment de la création de l'objet.

Constructeur avec un Seul Argument (String nom) :

- Initialise le nom avec la valeur fournie et age à 0.
- Utile lorsque l'on connaît seulement le nom de la personne.

Constructeur avec Deux Arguments (String nom, int age) :

- Initialise le nom et l'age avec les valeurs fournies.
- Utilisé quand on connaît à la fois le nom et l'âge de la personne.

Utilisation des Constructeurs Surchargés

Pour créer des objets Personne :

```
public class Main {
    public static void main(String[] args) {
        Personne personne1 = new Personne();
        Personne personne2 = new Personne("Alice");
        Personne personne3 = new Personne("Bob", 30);

        // Ici, vous pouvez utiliser des méthodes pour afficher les
informations de ces objets, etc.
    }
}
```

Dans le main, trois objets `Personne` sont créés en utilisant chacun des constructeurs surchargés. Cela montre comment on peut initialiser des objets de différentes manières selon les informations disponibles.

La surcharge de constructeur offre donc une flexibilité dans la création d'objets, permettant de définir plusieurs façons d'initialiser un objet en fonction des données disponibles ou des besoins spécifiques.

Définition de la portée des variables en Java

La portée d'une variable détermine où elle peut être utilisée dans votre programme.

Variables locales

Déclarées à l'intérieur d'une méthode, accessibles uniquement dans cette méthode.

Exemple de Code avec Variables Locales

Dans cet exemple, nous créerons une classe Calculatrice simple qui aura une méthode pour calculer la somme de deux nombres. Les variables utilisées pour stocker les nombres et le résultat seront des variables locales.

```
 public class Calculatrice {

    public void calculerSomme() {
        // Déclaration de variables locales
        int nombre1 = 5;
        int nombre2 = 10;
        int somme;

        // Calcul de la somme
        somme = nombre1 + nombre2;

        // Affichage du résultat
        System.out.println("La somme de " + nombre1 + " et " + nombre2 + "
est: " + somme);
    }

    public static void main(String[] args) {
        Calculatrice calc = new Calculatrice();
        calc.calculerSomme(); // Appel de la méthode qui utilise des
variables locales
    }
}
```

Dans ce code :

`nombre1`, `nombre2`, et `somme` sont des variables locales à la méthode `calculerSomme`.
Ces variables ne peuvent être utilisées qu'à l'intérieur de cette méthode.
Elles ne sont pas accessibles depuis d'autres méthodes, y compris la méthode main.

Variables d'instance

Les variables d'instance sont déclarées dans une classe mais à l'extérieur de toute méthode.
Elles sont accessibles par toutes les méthodes de la classe, et chaque instance de la classe a sa propre copie de ces variables.

Exemple de Code avec Variables d'Instance

Nous créerons une classe Personne avec des variables d'instance pour le nom et l'âge. La classe aura des méthodes pour définir et obtenir ces valeurs.

```
  public class Personne {
    // Variables d'instance
    private String nom;
    private int age;

    // Constructeur pour initialiser les variables d'instance
    public Personne(String nom, int age) {
        this.nom = nom;
        this.age = age;
    }

    // Méthodes getter et setter pour nom
    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    // Méthodes getter et setter pour age
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    // Méthode pour afficher les informations de la personne
    public void afficherInfos() {
        System.out.println("Nom: " + nom + ", Age: " + age);
    }

    // Méthode principale pour tester la classe
    public static void main(String[] args) {
        Personne personne1 = new Personne("Alice", 30);
        personne1.afficherInfos();

        Personne personne2 = new Personne("Bob", 25);
        personne2.setAge(26); // Modification de l'âge de Bob
        personne2.afficherInfos();
    }
}
```

Dans ce code :

`nom` et `age` sont des variables d'instance.
Chaque objet `Personne` a sa propre copie de ces variables.
Les méthodes `setNom`, `setAge`, `getNom`, `getAge`, et `afficherInfos` accèdent et modifient ces variables d'instance.
La méthode `main` crée des instances de `Personne` et appelle leurs méthodes pour illustrer comment les variables d'instance fonctionnent.

Variables de classe "static"

Les variables de classe statiques, aussi connues sous le nom de variables statiques, sont partagées entre toutes les instances d'une classe.
Elles sont déclarées avec le mot-clé `static`.

Exemple de Code avec Variables de Classe Statiques

Créons une classe Compteur qui utilise une variable de classe statique pour suivre le nombre total d'instances de cette classe créées.

```
public class Compteur {
    // Variable de classe statique
    private static int nombreTotalDeCompteurs = 0;

    // Constructeur qui incrémente la variable statique à chaque fois
qu'une nouvelle instance est créée
    public Compteur() {
        nombreTotalDeCompteurs++;
    }

    // Méthode statique pour obtenir la valeur de la variable statique
    public static int getNombreTotalDeCompteurs() {
        return nombreTotalDeCompteurs;
    }

    // Méthode principale pour tester la classe
    public static void main(String[] args) {
        Compteur compteur1 = new Compteur();
        Compteur compteur2 = new Compteur();
        Compteur compteur3 = new Compteur();

        System.out.println("Nombre total de compteurs créés: " +
Compteur.getNombreTotalDeCompteurs());
    }
}
```

Dans ce code

`nombreTotalDeCompteurs` est une variable de classe statique.
Elle est partagée par toutes les instances de la classe `Compteur`.
À chaque fois qu'un nouvel objet `Compteur` est créé, le constructeur incrémente `nombreTotalDeCompteurs`.
La méthode `getNombreTotalDeCompteurs` est également statique, permettant d'accéder à la variable statique sans avoir besoin de créer une instance de `Compteur`.
Dans la méthode `main`, trois instances de `Compteur` sont créées, et la valeur de `nombreTotalDeCompteurs` est affichée, montrant que la variable statique est partagée et mise à jour à travers toutes les instances.

Le mot clé 'this' en JAVA

Le mot-clé `this` en Java fait référence à l'objet courant.
C'est-à-dire, l'objet pour lequel la méthode ou le constructeur est appelé.

"This" pour distinguer les Variables d'Instance et les Paramètres de Méthode

Lorsqu'une méthode ou un constructeur a des paramètres avec les mêmes noms que les variables d'instance, `this` est utilisé pour différencier les variables d'instance des paramètres.

```
public class Membre {
    private String nom;  // variable d'instance
    private String prenom; // variable d'instance
    private int age; // variable d'instance

    public Membre(int age, string nom, string prenom) {
        this.age = age; // 'this.age' fait référence à la variable
d'instance et 'age' au paramètre de la methode
        this.nom = prenom; // 'this.nom' fait référence à la variable
d'instance et 'nom' au paramètre de la methode
        this.nom = prenom; // 'this.prenom' fait référence à la variable
d'instance et 'prenom' au paramètre de la methode
    }
}
```

A noter

- `this` est utilisé uniquement à l'intérieur des méthodes d'instance, des constructeurs, et des initialisateurs d'instance.
- Il ne peut pas être utilisé dans des méthodes statiques car une méthode statique appartient à la classe, pas à un objet spécifique.
- L'utilisation de `this` est facultative sauf dans les cas où un paramètre de méthode masque une variable d'instance.

Executer un programme JAVA avec la Méthode "main"

La méthode `main` est le point d'entrée de tout programme Java. Lorsque vous exécutez une application Java, c'est la méthode `main` qui est appelée en premier. Sans cette méthode, votre programme ne pourrait pas démarrer.

Structure de la Méthode main

Voici à quoi ressemble typiquement une méthode main en Java :

```
public class MonProgramme {
    public static void main(String[] args) {
        // Votre code ici
        System.out.println("Hello Word  !");
    }
}
```

Explication des Composants

public :

Un modificateur d'accès qui indique que la méthode main peut être appelée de n'importe où, y compris en dehors de sa classe ou de son package.

static :

Signifie que la méthode main peut être appelée sans créer d'instance de la classe **"MonProgramme"**. C'est nécessaire car lorsque votre programme démarre, il n'y a pas encore d'objet pour appeler des méthodes.

void :

Indique que la méthode `main` ne retourne rien.

main :

Le nom de la méthode. C'est un nom spécial que le JVM (Java Virtual Machine) recherche comme point de départ pour exécuter le programme.

String[] args :

Le paramètre **"args"** stocke les arguments de ligne de commande que vous pouvez passer lorsque vous exécutez votre programme. Il s'agit d'un tableau de chaînes de caractères (String). Par exemple, si vous exécutez votre programme avec java `MonProgramme arg1 arg2`, alors **"args[0]"** contiendra **"arg1"** et **"args[1]"** contiendra **"arg2"**.

Points Importants sur la methode "main"

- Chaque application Java doit avoir au moins une méthode main.
- Vous pouvez avoir plusieurs classes avec des méthodes main dans un même programme, mais l'exécution commence avec la méthode main de la classe que vous exécutez.
- La méthode main est le point de départ de l'exécution de votre programme et est essentielle pour tout programme Java.

Comment afficher du contenu à l'écran

En Java, il existe plusieurs façons d'afficher du texte ou le contenu de différentes types de variables à l'écran. Voici les méthodes les plus courantes :

Utilisation de System.out.println() et ses Variantes

`System.out.println()` est probablement la méthode la plus utilisée pour afficher des informations à l'écran. Elle affiche le texte ou la valeur de la variable, puis termine la ligne en cours (ajoute un saut de ligne).

```
String message = "Hello word !";
System.out.println(message);
```

`System.out.print()` est similaire, mais ne termine pas la ligne en cours. Il n'y a pas de saut de ligne

```
System.out.print("Bonjour ");
System.out.print("le monde!");
```

`System.out.printf()` permet un formatage plus complexe en ajoutant des variables

```
int age = 30;
System.out.printf("Age: %d ans\n", age);
```

Syntaxe de base de printf

La méthode printf suit généralement la syntaxe suivante :

```
System.out.printf("format-string", argument1, argument2, ...);
```

- **format-string :** Une chaîne de caractères qui spécifie le format (Spécificateurs de Format).
  Elle peut contenir des littéraux de texte mélangés à des **spécificateurs de format (%x)**.
- **arguments :** Les variables (valeurs) qui doivent être formatées et insérées dans la chaîne de format.

Spécificateurs de Format

Les spécificateurs de format commencent par un signe de pourcentage **%** et se terminent par un caractère qui indique le type de la variable.
Voici quelques spécificateurs de format courants :

- **%s :** pour une chaîne de caractères.
- **%d :** pour un entier décimal.
- **%f :** pour un nombre à virgule flottante.
- **%n :** pour un saut de ligne (équivalent à \n).
- **%t :** pour les formats de date/heure.

```
String name = "Alice";
int age = 25;
System.out.printf("Nom: %s, Âge: %d%n", name, age);
```

Cela affichera **"Nom: Alice, Âge: 25"**.

La gestion des Packages en JAVA

Un "package" en Java est un moyen d'organiser les fichiers de votre code source Java. C'est comme un dossier dans un système de fichiers de votre ordinateur. Les packages sont utilisés pour regrouper des classes Java liées fonctionnellement, ce qui rend votre code plus organisé, facile à gérer et à comprendre.

Pourquoi Utiliser des Packages ?

Eviter les Conflits de Noms :

Les packages permettent de catégoriser les classes. Deux classes peuvent avoir le même nom tant qu'elles sont dans des packages différents.

Contrôle de l'Accessibilité :

Ils aident à contrôler l'accès aux classes et aux membres de classe grâce à la visibilité au niveau des packages.

Facilité de Maintenance :

Un code bien organisé en packages est plus facile à maintenir et à comprendre.

Comment Créer un Package ?

Pour créer un package, vous utilisez la déclaration **"package"** au début de votre fichier source Java.

```
package com.monentreprise.monapplication;
```

Cette ligne doit être la première instruction (hors-commentaire) dans le fichier.
Elle indique que toutes les classes définies dans ce fichier appartiennent au package com.monentreprise.monapplication.

Structure des Packages

Les noms de packages sont souvent structurés de manière hiérarchique, similaire aux repertoire/fichier ou domaines Internet, pour assurer l'unicité.

Dans le cas des noms de domaines nous utiliserons la structure de **nom de domaine inversée** (com.nomdomaine.nomprojet)

- monprojet.monsouspackage.monpackage pour des classes dédiées à mon projet
- com.masociete.projet pour des classes appartenant à une organisation.

Utilisation des Packages

Pour utiliser une classe d'un autre package, vous avez deux options :

Utilisez le nom complet de la classe, y compris le nom du package :

```
public class MaClasse {

  public static void main(String[] args) {
    int[] tableau = {5, 6, 3, 4};
    java.util.Arrays.sort(tableau);
  }
}
```

Par Importation du package :

```
import java.util.Arrays //importation du package contenant les classes

public class MaClasse {
  public static void main(String[] args) {
    int[] tableau = {5, 6, 3, 4};
```

```
    sort(tableau); //la classe dont le package est importé en début de
fichier
  }
}
```

Utilisez l'instruction import pour importer une classe ou toutes les classes d'un package.

```
import java.util.ArrayList;
import java.util.*;
```

Si vous avez un fichier **"MaClasse.java"** dans le package **"com.monentreprise.util"**, la structure sera :

```
package com.monentreprise.util;

public class MaClasse {
    // Définitions de la classe
}
```

Pour utiliser **"MaClasse"** dans un autre fichier :

```
import com.monentreprise.util.MaClasse;

public class Test {
    MaClasse maClasse = new MaClasse();
}
```

Convention de nommage des packages

Utiliser la Notation en Minuscules

Les noms de packages devraient toujours être en minuscules pour éviter les conflits sur des systèmes dont la distinction entre majuscules et minuscules est importante.

- Correct : com.monentreprise.projet
- Incorrect : com.MonEntreprise.Projet

En Conclusion sur les packages

Les packages en Java sont un outil puissant pour organiser votre code. Ils aident à éviter les conflits de noms, facilitent la gestion de l'accessibilité et améliorent la lisibilité et la maintenance du code. Comprendre et utiliser correctement les packages est une compétence essentielle pour tout développeur Java.

Conclusion sur les basiques de JAVA

JAVA et l'Indépendance de la Plate-forme

L'indépendance de la plate-forme signifie que le code écrit en Java peut être exécuté sur n'importe quelle machine, peu importe son système d'exploitation ou son architecture matérielle.
Ceci est rendu possible grâce à la machine virtuelle Java (JVM).

Avantages : Cette caractéristique permet une grande flexibilité et portabilité du code. Un programme écrit sur un système d'exploitation peut être exécuté sur un autre sans modification.

Comment cela fonctionne : Le code Java est compilé en un format intermédiaire appelé bytecode, qui est ensuite interprété ou compilé en temps réel par la JVM sur la machine cible.

L'Orientation Objet

Java est un langage de programmation orienté objet, ce qui signifie que tout dans Java est associé à des classes et des objets.

Classes : Des modèles ou des plans à partir desquels les objets sont créés.

Objets : Des instances de classes qui ont des états (attributs) et des comportements (méthodes).

Avantages : L'orientation objet aide à structurer les programmes de manière plus intuitive, en facilitant la maintenance, le développement et la compréhension du code.

La notion d'Encapsulation

L'encapsulation est un principe de la programmation orientée objet qui consiste à masquer les détails d'implémentation d'une classe et à exposer uniquement des méthodes de manipulation sûres.

Comment ça fonctionne : En Java, cela est généralement réalisé en utilisant des modificateurs d'accès comme **private**, **protected**, et **public**.

Avantages : Elle permet de protéger l'intégrité des données d'un objet en évitant l'accès direct aux attributs (variables) de la classe et en permettant la manipulation des données uniquement via des méthodes définies.